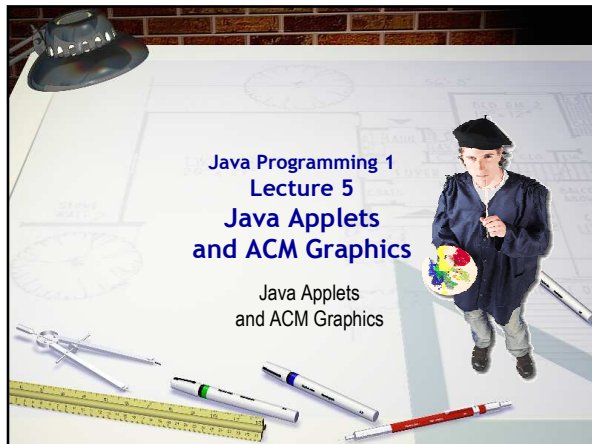




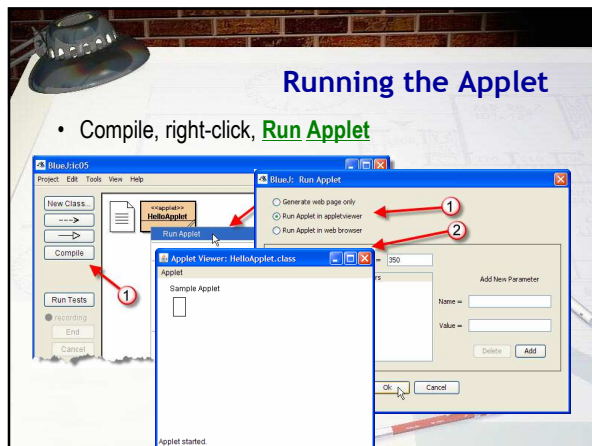
Express Yourself

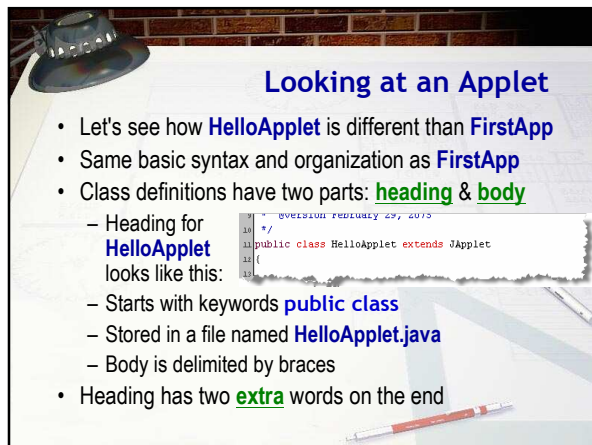
- Use OpenOffice Writer to create a new document
- Save the file as LastFirst_ic05
 - Replace LastFirst with your actual name
- Put your name and today's date at the top of the sheet
- Title it "CS 170 In-class Exercise 5"
- **Exercise 1:** Start BlueJ and create a new project named ic05. Create a new class named HelloApplet using the Swing Applet template. Shoot me a screen-shot. (Step-by-step pics on next slide.)

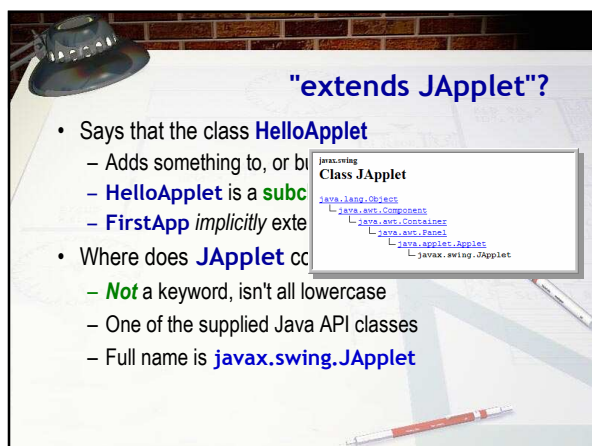
Your First Applet

- Click the **New Class** button
- Name: **HelloApplet**
- Template: **Swing Applet**
- **OK**

Two screenshots of the BlueJ IDE. The left screenshot shows the 'New Class' button in the 'Project' window, with a red arrow pointing to it and the text 'Open in the editor'. The right screenshot shows the 'Create New Class' dialog box with 'Class Name' set to 'HelloApplet' and 'Class Type' set to 'Swing Applet'. Red arrows and numbers 1 through 4 indicate the steps: 1. 'New Class' button, 2. 'Class Name' field, 3. 'Swing Applet' template, 4. 'OK' button.







Class Body

- Class body is delimited by braces `{ }` (begin...end)
- Inside the class body, you'll define two things:
 - **Attributes** -- the data fields used to store object state
 - **Methods** -- where the actions take place in an object
- **HelloApplet** already has one sample field defined
 - We won't use it for now, so just delete it.

```
1 public class HelloApplet extends JApplet
2 {
3     // Instance/static variables .....
4     private int x; /*# replace with your own variable(s) */
5
6     /**
7      * Called for the browser to implement the information. Applet class
8      */
9 }
```

Delete this line for now

Redefining Methods

- An object's methods are where actions occur
 - **HelloApplet** is a "child" of **JApplet**, so it **inherits** several methods from its parent and grandparents
 - These five methods are **automatically** called:
 - **init()**, **start()**, **stop()**, **destroy()**, and **paint()**
 - The inherited versions don't do anything interesting
 - We can **override** or **redefine** these methods
- **HelloApplet** overrides **paint()** and **init()** methods
 - The **init()** method includes code for a browser bug

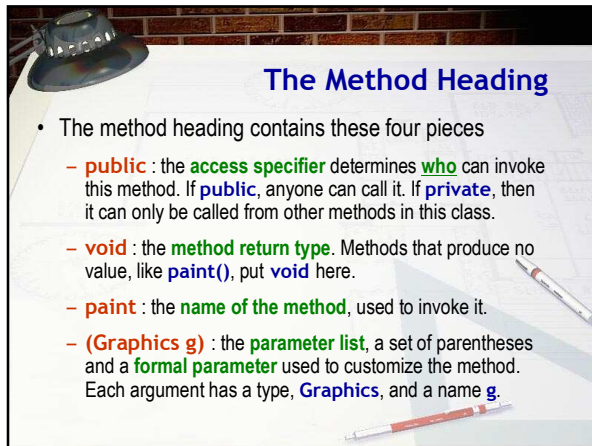
Defining a Method

- Methods, remember, are defined **inside** your class
 - Consist of a heading and body, just like a class
 - Template uses both simple and advanced graphics

```
1 public void paint(Graphics g)
2 {
3     // simple text displayed on applet
4     g.setColor( Color.black );
5     g.drawString( "Sample Applet", 20, 20 );
6
7     // Graphics2D for advanced drawing. (Horstmann, p. 64)
8     Graphics2D g2 = (Graphics2D) g;
9
10    Rectangle box = new Rectangle(25, 30, 20, 30);
11    g2.draw(box);
12 }
13
```

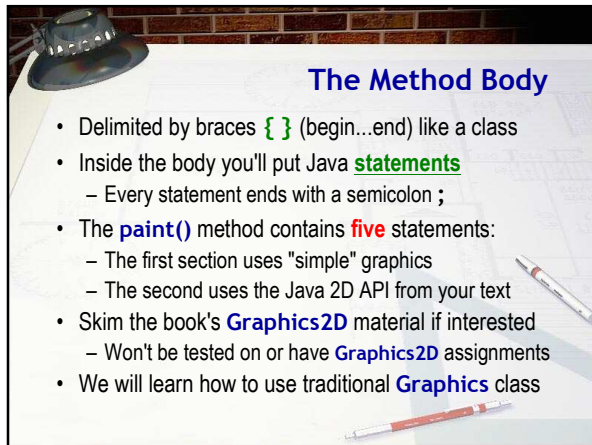
Simple graphics

Advanced graphics



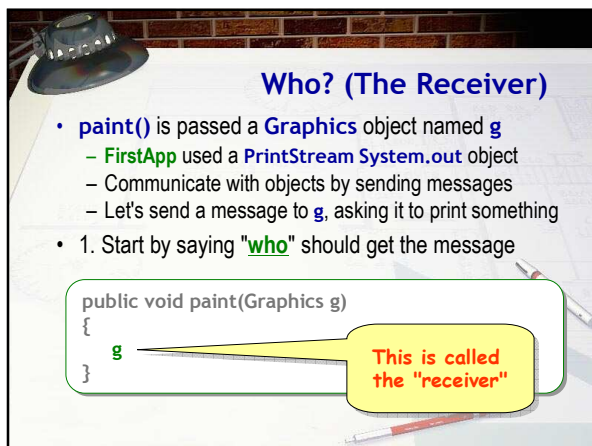
The Method Heading

- The method heading contains these four pieces
 - public**: the **access specifier** determines **who** can invoke this method. If **public**, anyone can call it. If **private**, then it can only be called from other methods in this class.
 - void**: the **method return type**. Methods that produce no value, like **paint()**, put **void** here.
 - paint**: the **name of the method**, used to invoke it.
 - (Graphics g)**: the **parameter list**, a set of parentheses and a **formal parameter** used to customize the method. Each argument has a type, **Graphics**, and a name **g**.



The Method Body

- Delimited by braces **{ }** (begin...end) like a class
- Inside the body you'll put Java **statements**
 - Every statement ends with a semicolon **;**
- The **paint()** method contains **five** statements:
 - The first section uses "simple" graphics
 - The second uses the Java 2D API from your text
- Skim the book's **Graphics2D** material if interested
 - Won't be tested on or have **Graphics2D** assignments
- We will learn how to use traditional **Graphics** class



Who? (The Receiver)

- paint()** is passed a **Graphics** object named **g**
 - FirstApp** used a **PrintStream System.out** object
 - Communicate with objects by sending messages
 - Let's send a message to **g**, asking it to print something
- 1. Start by saying "**who**" should get the message

```
public void paint(Graphics g)
{
    g
}
```

This is called the "receiver"

What? (The Request)

- 2. Next, tell the object **what** to do

```
public void paint(Graphics g)
{
    g.drawString
}
```

This is called the "request"

- Separate receiver and request with a "dot"

Parameters

- 3. Add parentheses, and pass any **parameters**

```
public void paint(Graphics g)
{
    g.drawString("Hello, I love you!", 20, 20);
}
```

Add parentheses, parameters, semicolon

- What parameters?
Read the JavaDocs!
 - Arguments, receiver, must match documentation

Message Recap

- Receiver**: who the message goes to
- Request**: what you want the object to do
- Parameters**: additional needed information

```
someObject.doSomething( someInfo );
```

Receiver Request (Message) Parameters

Deciphering the Docs

- Read the method docs to "talk" to your objects
 - Locate the method summary

Method Summary

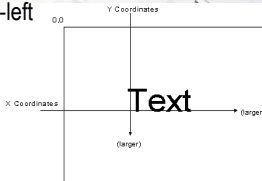
clearRect(int x, int y, int width, int height)
 Clears the specified rectangle by filling it with the background color of the current drawing surface.

drawString(String str, int x, int y)
 Draws the text given by the specified string, using this graphics context's current font and color.

1 2 3 4

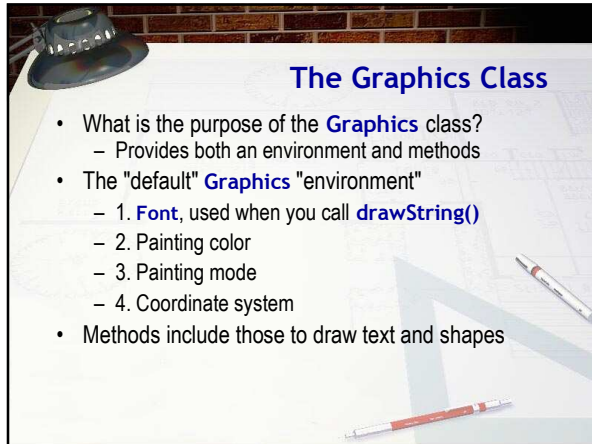
The drawString() Method

- The **drawString()** method takes three arguments:
 - A **String** containing the message to display
 - A **String** is "text between double-quotes"
 - Two **int** for the horizontal and vertical position
 - An **int** is just a plain whole number
- Coordinates** start in upper-left position (0,0) called **origin**
- x** increases to right
- y** increases down



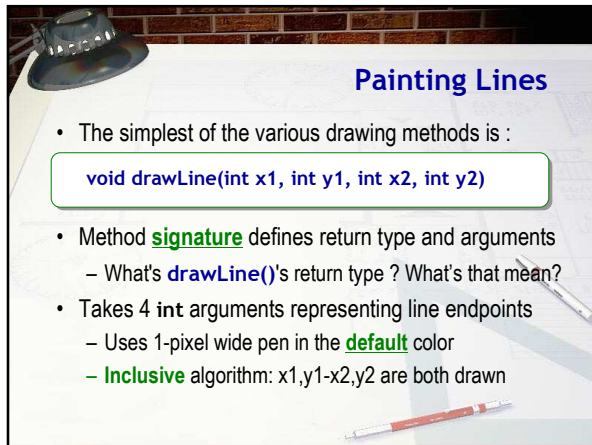
Express Yourself

- Exercise 2:** Delete all of the code inside HelloApplet's **paint()** method. Then use the **drawString** method to display the phrase: "Hello, my name is Steve", "I think I Love You" on two lines, one above the other, centered in your applet. Replace my name with your name, of course. Show me two screenshots: one of your program running, and one of your **paint()** code.
 - Hint: you'll need to adjust the x, y arguments using trial and error. There is no "center this line" command.



The Graphics Class

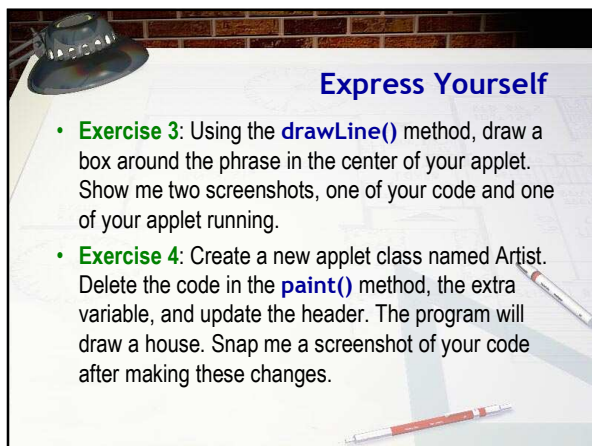
- What is the purpose of the **Graphics** class?
 - Provides both an environment and methods
- The "default" **Graphics** "environment"
 - 1. **Font**, used when you call **drawString()**
 - 2. Painting color
 - 3. Painting mode
 - 4. Coordinate system
- Methods include those to draw text and shapes



Painting Lines

- The simplest of the various drawing methods is :

`void drawLine(int x1, int y1, int x2, int y2)`
- Method **signature** defines return type and arguments
 - What's **drawLine()**'s return type ? What's that mean?
- Takes 4 **int** arguments representing line endpoints
 - Uses 1-pixel wide pen in the **default** color
 - **Inclusive** algorithm: x1,y1-x2,y2 are both drawn



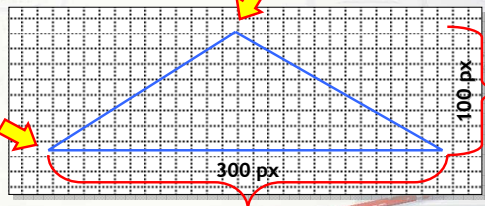
Express Yourself

- **Exercise 3:** Using the **drawLine()** method, draw a box around the phrase in the center of your applet. Show me two screenshots, one of your code and one of your applet running.
- **Exercise 4:** Create a new applet class named Artist. Delete the code in the **paint()** method, the extra variable, and update the header. The program will draw a house. Snap me a screenshot of your code after making these changes.

Exercise 5
Show me your code
Snap a pic of running applet

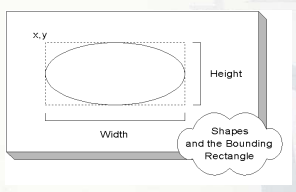
Hands On:

- Draw a triangle 100 pixels high, with a base of 300
- Center the apex over the base
 - Put the triangle 10 from the left, 10 down from the top



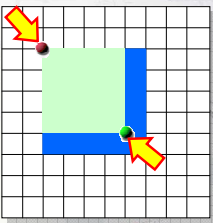
Drawing Closed Shapes

- Could use same 4 args to draw rectangles or ovals
 - **Instead**, Java's shapes use a different "bounding box"
 - Uses **width** and **height** instead of x2, y2



Drawn versus Filled Shapes

- Coordinates used for x,y represent "points"
 - Infinitely small location located between pixels
 - Example: points 2,2 and 6,6
- Shape-drawing methods come in two varieties
 - Those that **draw** an outline
 - Use 1-pixel wide pen that hangs down/right
 - Those that **fill** an area
- Drawn shapes are 1-pixel bigger than filled shapes



Plain Rectangles

- Three methods:

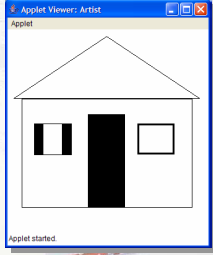

```
void drawRect(int x, int y, int width, int height)
void fillRect(int x, int y, int width, int height)
void clearRect(int x, int y, int width, int height)
```

 - First draws outline anchored at x,y in foreground color
 - Will be one pixel higher and wider than requested
 - Second draws solid square in foreground color
 - Last works like `fillRect()`, using background color

Hands On: Exercise 6

Show me your code
Snap a pic of running applet

- Let's put this to work by drawing a "house" underneath the "roof" we've already created
 - Use `drawRect()` to paint a rectangle 275 pixels wide and 175 pixels high under the roof
 - Use `fillRect()` to paint a door in the middle of the house
 - What would you use to paint a window, like this?
 - How about this?

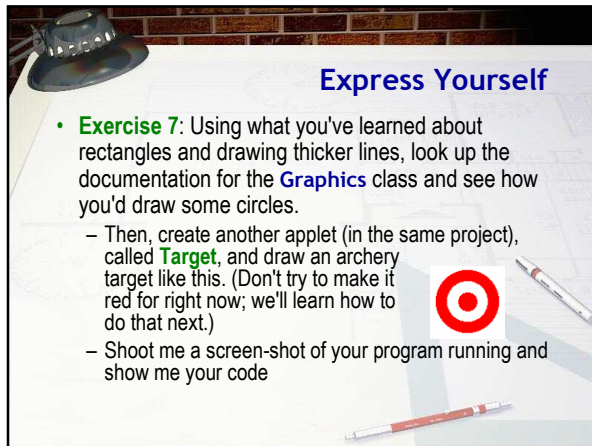


Drawing Thicker Lines

- Issue a **sequence** of `drawRect()` calls
 - Each time, increase `x`, `y` by 1
 - Decrease `width` and `height` by 2

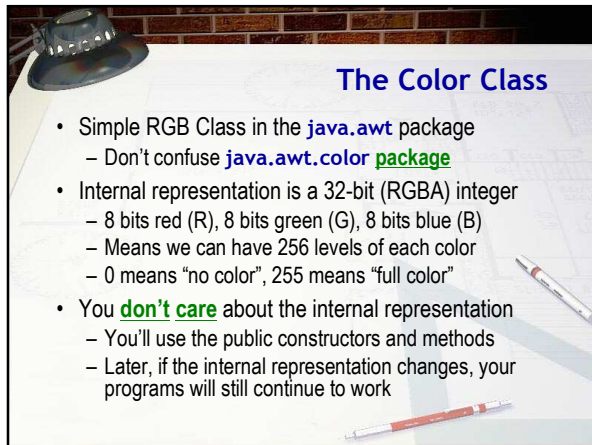
```
g.drawRect(10, 10, 100, 100);
g.drawRect(11, 11, 98, 98);
g.drawRect(12, 12, 96, 96);
```
- Use a combination of `fillRect()` and `clearRect()`

```
g.fillRect(10, 10, 100, 100);
g.clearRect(20, 20, 80, 80);
```



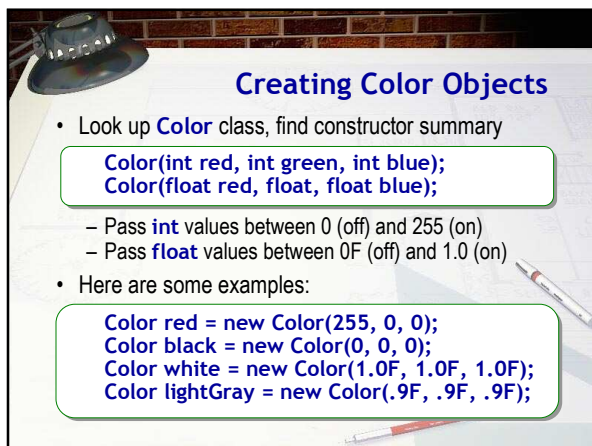
Express Yourself

- **Exercise 7:** Using what you've learned about rectangles and drawing thicker lines, look up the documentation for the **Graphics** class and see how you'd draw some circles.
 - Then, create another applet (in the same project), called **Target**, and draw an archery target like this. (Don't try to make it red for right now; we'll learn how to do that next.)
 - Shoot me a screen-shot of your program running and show me your code



The Color Class

- Simple RGB Class in the **java.awt** package
 - Don't confuse **java.awt.color** package
- Internal representation is a 32-bit (RGBA) integer
 - 8 bits red (R), 8 bits green (G), 8 bits blue (B)
 - Means we can have 256 levels of each color
 - 0 means "no color", 255 means "full color"
- You **don't care** about the internal representation
 - You'll use the public constructors and methods
 - Later, if the internal representation changes, your programs will still continue to work



Creating Color Objects

- Look up **Color** class, find constructor summary
 - Color(int red, int green, int blue);**
 - Color(float red, float, float blue);**
 - Pass **int** values between 0 (off) and 255 (on)
 - Pass **float** values between 0F (off) and 1.0 (on)
- Here are some examples:
 - Color red = new Color(255, 0, 0);**
 - Color black = new Color(0, 0, 0);**
 - Color white = new Color(1.0F, 1.0F, 1.0F);**
 - Color lightGray = new Color(.9F, .9F, .9F);**

The setColor() Method

- **setColor()** changes the current painting color
- Use with your **Color** variable like this:

```
public void paint(Graphics g)
{
    Color brush = new Color(255, 128, 128);
    g.setColor(brush);
}
```

- **Exercise 8:** Make your target red. Snap a picture.
- **Exercise 9:** Put green windows and a red door on your artist's cottage. Snap a picture, show me your code.

Other Color Constructors

- Modern browsers can also display transparent colors

```
Color(int red, int green, int blue, int alpha);
```

- The alpha channel is 0 (transparent) to 255 (opaque)

- You can also specify colors using a single integer

```
Color(int colorValue);
```

- Mostly useful when working with HTML Hex colors

Color Class Constants

- Objects and variables take up space in memory

```
Color r1 = new Color(255, 0, 0);
Color r2 = new Color(255, 0, 0);
```

- Takes up room for two variables and two objects

- **Class constants:** predefined values stored in class

- Accessed using **class** name, rather than **object** name

```
Color r1 = Color.red;
Color r2 = Color.red;
g.setColor(Color.lightGray);
```

Express Yourself

- Here's one of the **Color** constructors you just studied:

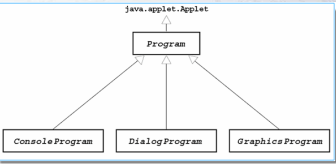
```
Color(int red, int green, int blue);
```
- Exercise 10:** Write a statement that creates a **Color variable** named **background**. Don't initialize it.
- Exercise 11:** Set the variable **background** to the color green, using the constructor shown here.
- Exercise 12:** Change **background** to a medium gray. Do it both with and without using a constructor

The ACM Class Libraries

- The ACM Java libraries were designed for students
 - ACM: Association for Computing Machinery
 - Computer Science Professional and Academic Organization
 - Student membership is about \$20 per year
 - <http://www.acm.org>
- Java Task Force (JTF) – Eric Roberts (Stanford) chair
 - Review the Java language
 - Develop a stable collection of libraries for teaching
 - <http://jtf.acm.org>
- We'll use several classes from the library

The ACM Program Classes

- Problem 1:** Java
 - Can stick a **main** method in a class
 - Can't access in a class
 - Not very object-oriented
- Problem 2:** Applets
 - Can't run as applications
 - Can't change input style by changing a single line of code
 - Both console-mode and graphics versions
- Solution:** ACM program classes
 - Simple and OO like applets
 - Can be run as both applets or applications
 - Change input style by changing a single line of code
 - Both console-mode and graphics versions



```
graph TD
    Applet["java.applet.Applet"] --> Program["Program"]
    Program --> ConsoleProgram["ConsoleProgram"]
    Program --> DialogProgram["DialogProgram"]
    Program --> GraphicsProgram["GraphicsProgram"]
```

The ACM Graphics Classes

- Applet graphics are not particularly object-oriented
 - Used
 - Preci
 - Relia
 - is rea
- Want to
 - Easy
 - We'll start with the ACM **GraphicsProgram** class, and the components in the **acm.graphics** package

The HelloACM Program

- We'll use the **GLabel** class to display "Hello, I love you!"
- Step 1:** Add a new class
 - Name: **HelloACM**
 - Type: **ACM Graphics**
- Step 2:** Open file

ACM Hello Changes

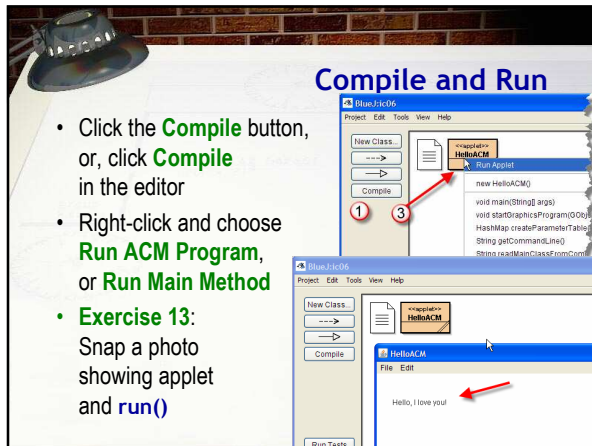
- Notice differences from Java Applet
 - Different imports: **acm.program**, **acm.graphics**
 - Extends **GraphicsProgram** instead of **Applet**
 - A **run()** method instead of a **main()** method
- Create a **GLabel** and add it to the applet at 25, 50

```

16  */
17  public void run()
18  {
19      GLabel helloLabel = new GLabel("Hello, I love you!");
20      add(helloLabel, 25, 50);
21  }
22
  
```

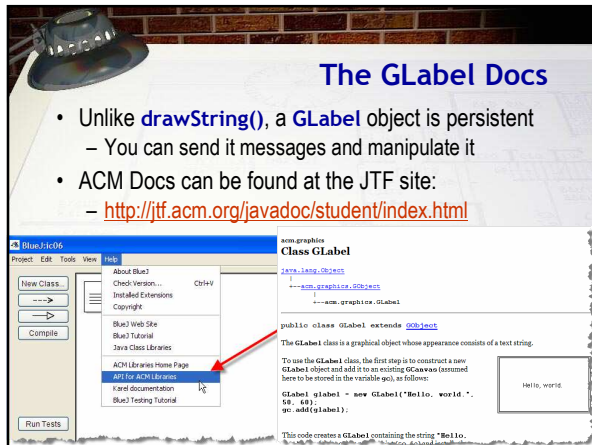
Compile and Run

- Click the **Compile** button, or, click **Compile** in the editor
- Right-click and choose **Run ACM Program**, or **Run Main Method**
- Exercise 13:** Snap a photo showing applet and **run()**



The GLabel Docs

- Unlike **drawString()**, a **GLabel** object is persistent
 - You can send it messages and manipulate it
- ACM Docs can be found at the JTF site:
 - <http://jtf.acm.org/javadoc/student/index.html>



Font, Color, and Label

Exercise 14
Use your name, choose your font & color. Snap

- Use a font descriptor: **type-weight-size**

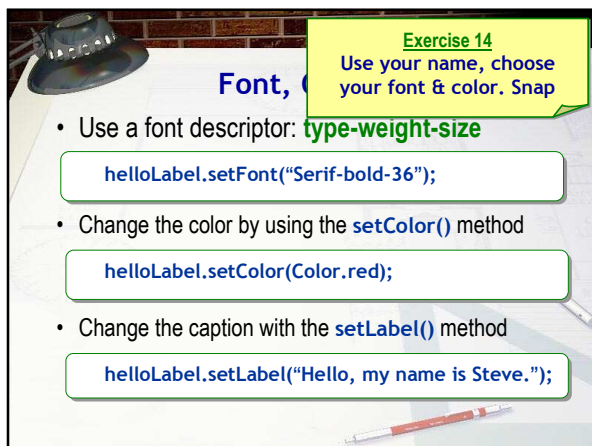
```
helloLabel.setFont("Serif-bold-36");
```

- Change the color by using the **setColor()** method

```
helloLabel.setColor(Color.red);
```

- Change the caption with the **setLabel()** method

```
helloLabel.setLabel("Hello, my name is Steve.");
```

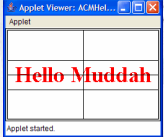


Changing Position

- The **GLabel** origin is the baseline of the text printed
 - Kind of unusual, but it's how **drawString()** works



- Centering text was hard with **drawString()**. It's easier with the **GLabel** class



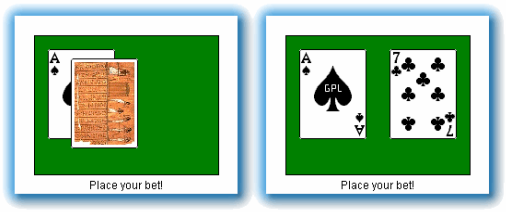
Changing Position

- Change position using **setLocation()**
- Use **getWidth()** and **getHeight()** for size
 - Can center your labels using code like this:

```
double w = this.getWidth(); // or omit the word this
double h = getHeight();
double wLbl = helloLabel.getWidth();
double hLbl = helloLabel.getHeight();
helloLabel.setLocation(w/2 - wLbl/2, h/2 + hLbl/4);
```

The TwoCards Applet

- Let's sample some of the other ACM graphics classes
 - Create a new class called TwoCards
 - Follow steps from ACMHello



1A. Create the Table

- We'll use the **GRect** class for the table
 - Define two constants for width and height
 - Create a **GRect** using 0,0 and **T_WIDTH** and

```
// 1a. Create a green "table" for our cards
final int T_WIDTH = 200;
final int T_HEIGHT = 150;
GRect table = new GRect(0, 0, T_WIDTH, T_HEIGHT);
```

1B. Center the Table

- Center the table like you centered your **GLabel**

```
// 1a. Center the table
int appWidth = getWidth();
int appHeight = getHeight();
table.setLocation(appWidth/2 - T_WIDTH/2,
                 appHeight/2 - T_HEIGHT/2);
```

1C. Color the Table

- **GRect** is a "fillable" class—has optional background
 - Specify the color using **setFillColor()**
 - Turn filling "on" using **setFilled()**
 - Add the table to the surface of the app

```
// 1b. Fill it with a dark green color
table.setFillColor(new Color(0, 128, 0));
table.setFilled(true);
add(table);
```

1D. Caption the Table

- Position a **GLabel** underneath the table
 - Create the label and calculate its size
 - Use **setLocation()** to position it
 - Add it to the surface of the application

```
// 1c. Let's label it as well
GLabel caption = new GLabel("Place your bet!");
double capWidth = caption.getWidth();
double capHeight = caption.getHeight();
caption.setLocation(appWidth/2 - capWidth/2,
    appHeight/2 + T_HEIGHT/2 + capHeight);
add(caption);
```

Getting the Card Images

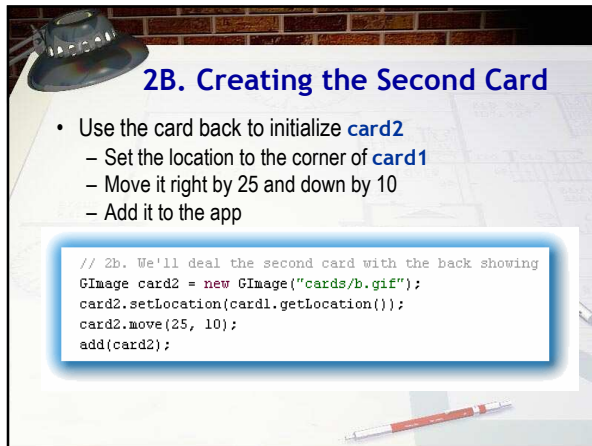
- Card images are generally copyrighted
 - Can't just scan in the cards from a pack of Bicycles
- One handy source of GPL card images
 - <http://www.waste.org/~oxymoron/cards>
 - Can use without charge; replace for commercial use
- Unzip images and store in folder named **cards** in same directory as your Java source code
 - Files named 2c.gif, as.gif, tc.gif, etc.
 - Back is b.gif



2A. Creating the First Card

- We'll use the **GImage** class to create our cards
 - Use the "Ace of Spades" to initialize the first card
 - Set its location in the corner of the table
 - Move it to the right by 15 pixels
 - Add it to the app

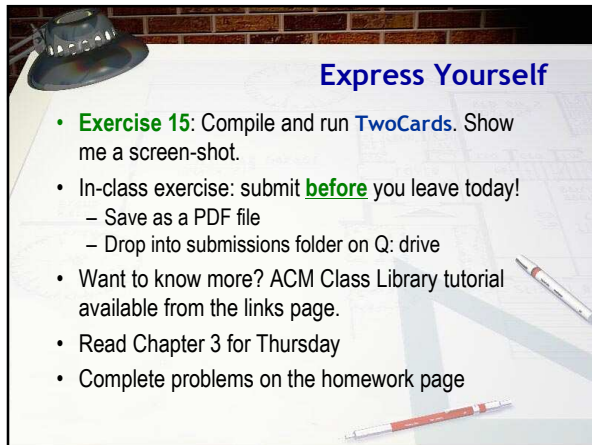
```
// 2. Let's create a card using the GImage class
GImage card1 = new GImage("cards/as.gif");
card1.setLocation(table.getLocation());
card1.move(15, 15);
add(card1);
```

A slide titled "2B. Creating the Second Card" with a background image of a desk with a lamp and a pen. The slide contains a bulleted list and a code block.

2B. Creating the Second Card

- Use the card back to initialize `card2`
 - Set the location to the corner of `card1`
 - Move it right by 25 and down by 10
 - Add it to the app

```
// 2b. We'll deal the second card with the back showing
GImage card2 = new GImage("cards/b.gif");
card2.setLocation(card1.getLocation());
card2.move(25, 10);
add(card2);
```

A slide titled "Express Yourself" with a background image of a desk with a lamp and a pen. The slide contains a bulleted list.

Express Yourself

- **Exercise 15:** Compile and run `TwoCards`. Show me a screen-shot.
- In-class exercise: submit **before** you leave today!
 - Save as a PDF file
 - Drop into submissions folder on Q: drive
- Want to know more? ACM Class Library tutorial available from the links page.
- Read Chapter 3 for Thursday
- Complete problems on the homework page
